

Unitree Week 19

Presentation by Mario Ortiz

This Week

- Research on Imitation Learning Pipeline Development
- Research on ideas for Unitree R1 Lab Course/Club
- Implementation of online datasets for Imitation Learning

Imitation Learning Pipelines: CNRS-AIST JRL

1. Collecting human motion data (using a motion capture system,
2. Obtain physical plausibility and compatibility with the robot's model
 - a. QP-based Inverse Kinematics: enforces joint limits and self-collision constraints
 - b. Pre-train an expert bipedal walking policy to guide leg motion during behavioral cloning.
 - c. Student policy: imitates the upper body and root motion for manipulation; walking expert policy: guides legs for locomotion.
3. Deployment:
 - a. Domain randomization used for sim-to-real gap
 - b. Motion capture: used to track the robot, box, and drop-off table

How to integrate

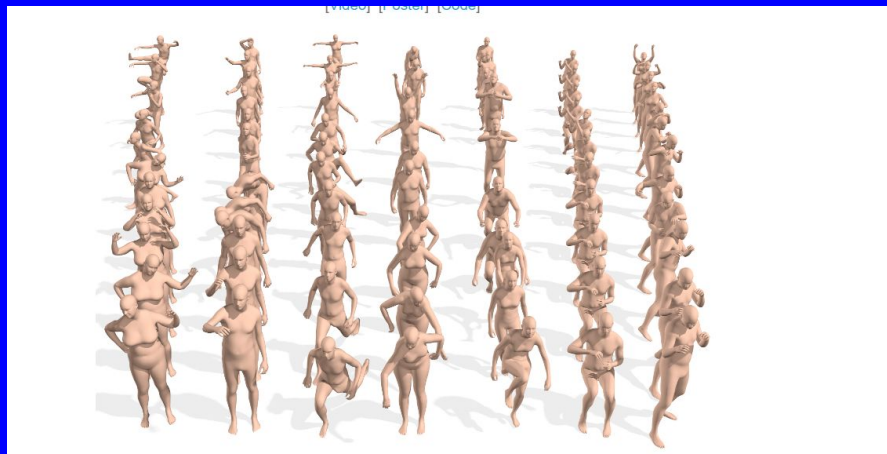
- Reinforcement Learning on Isaac Lab can be utilized for training the unitree G1/R1 for the walking policy.
- Two things are important to develop: Methodology for obtaining human motion data and QP-based Inverse Kinematics.

Human Motion Data

- Pre-existing datasets provide already made human motion data in which we can utilize at this moment.
 - AMASS:
 - MotionMillion
- Vision-based AI tools:
 - Rokoko Vision
 - FreeMoCap

AMASS

- Will attempt to use this for the human motion data.
- AMASS provides many components:
 - Full-body trajectories
 - Consistent SMPL body representation
 - Long temporal sequences
 - Realistic motion priors
- How to utilize: We will obtain an AMASS dataset and translate it into Unitree G1



Downloaded CMU (SMPLT+H G) dataset

- The CMU Motion Capture Database contains general-purpose movements (walking, running, jumping, turning, sports, natural transitions, etc.)

	Subjects	Motions	Minutes	Required Citation	License	Bodies	
CMU	96	1983	543.49	Citation	License	SMPL+H G	SMPL-X G
						SMPL-X N	Render

Download SMPL+H Model

- This model is necessary to convert AMASS pose parameters into actual 3D joints.

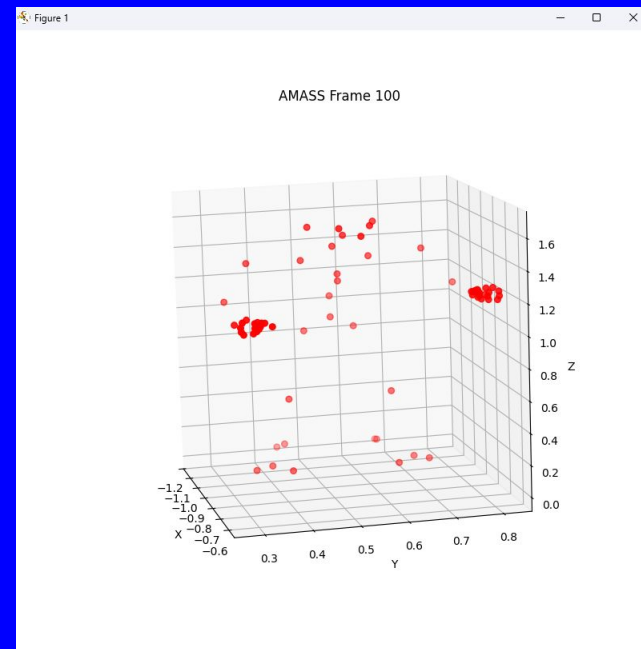
- [SMPLH model](#) version ready to load by the [smplx python package](#)
 - SMPL+H model in the format expected by the [smplx code repository](#) and [aitviewer](#). Both libraries can be used to load the [AMASS SMPL+H G](#) sequences.
 - 16 beta shape space components
 - Contains the following files:

```
smplx/  
└─ smplh/  
    ├── SMPLH_FEMALE.pkl  
    └── SMPLH_MALE.pkl
```

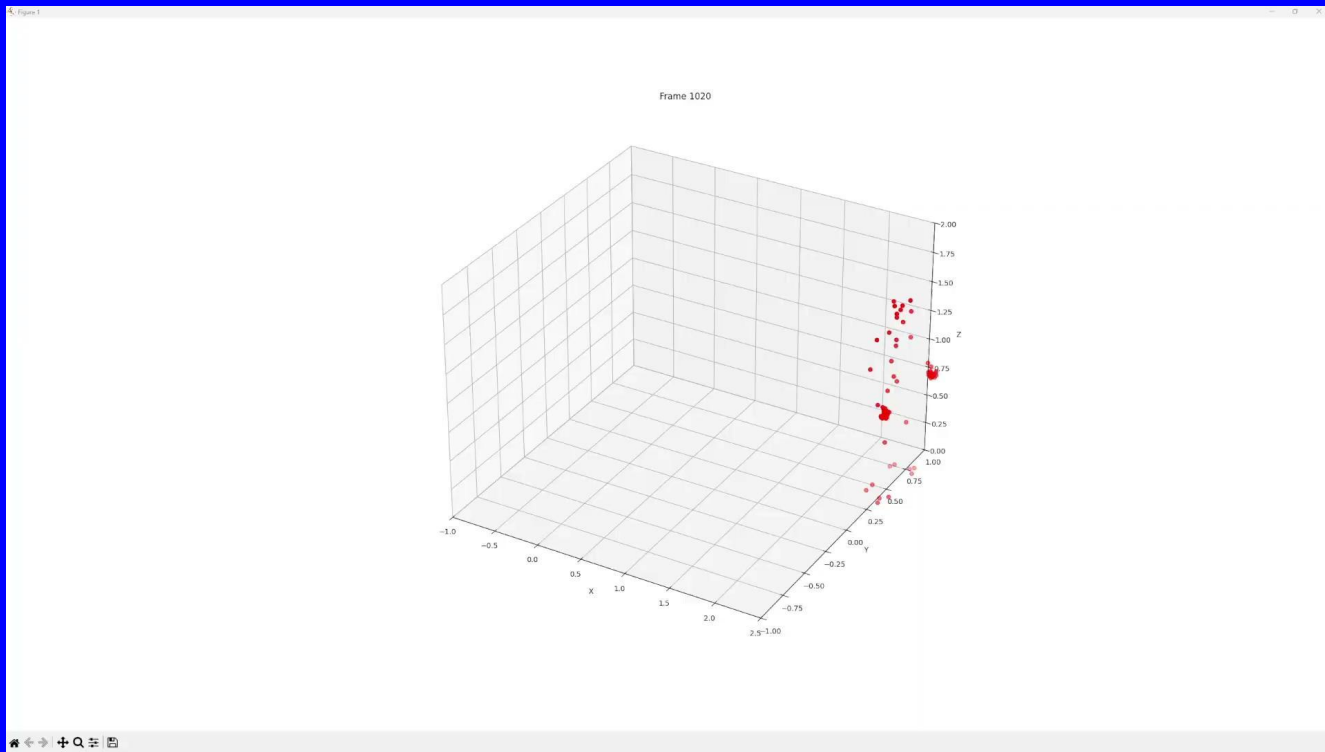
[Full SMPL+H model version with 200 shape components \(240MB, 00000544\)](#)

Using AMASS with SMPL+H

- First, we load the AMASS CPU motion capture data (contains pose, translation, body shape, and gender information which are extracted).
- Next we load and create the SMPL-H gender model to convert pose rotations and translations into 3D body joint positions.
- We run the motion data through

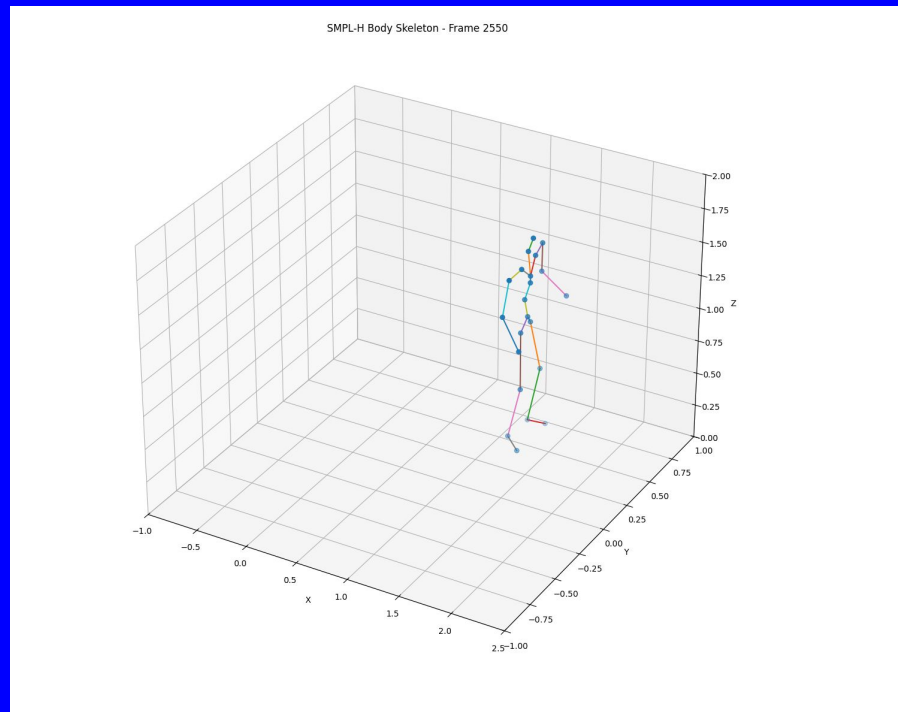


Using AMASS with SMPL+H



Skeletal Connections

- Afterwards we define skeletal connections to indicate what joints are connected to each other.
- Finally, the first 3D frame is made.
 - We update the frame and this is done several times.



Extract SMPL keypoints

- A program was develop to extract parameters that will be used for robot retargeting.
- We save this information into a .npz dataset file.
 - pelvis, feet, hands, head, knees, ankles, shoulders, ..., fps, foot contacts, root velocity

Preparing G1 Targets

- Next step is converting the SMPL human keypoints into Unitree G1 target positions.
 - Recentering the pelvis
 - Scaling the human body to the G1 size
 - Converting the AMASS axes to Isaac/G1 axes
 - Raising pelvis to the G1's height.
- base, left foot, right foot, .., head, contact, fps

G1 Model from Isaac Lab

- Using the G1 model's .usd file, we obtained the robot's joint names and body/link names. The exact naming conventions are used for retargeting purposes.

```
388 G1_2000F_Cfg = ArticulationCfg(  
389     spawn=sim_utils.UsdFileCfg(  
390         usd_path=f"[ISAAC_NUCLEUS_DIR]/Robots/Unitree/G1/g1.usd",  
391         activate_contact_sensors=False,  
392         rigid_props=sim_utils.RigidBodyPropertiesCfg(  
393             disable_gravity=False,  
394             retain_accelerations=False,  
395             linear_damping=0.0,  
396             angular_damping=0.0,  
397             max_linear_velocity=1000.0,  
398             max_angular_velocity=1000.0,  
399             max_depenetration_velocity=1.0,  
400         ),  
401         articulation_props=sim_utils.ArticulationRootPropertiesCfg(  
402             enabled_self_collisions=False,  
403             fix_root_link=False, # Configurable - can be set to True for fixed base  
404             solver_position_iteration_count=8,  
405             solver_velocity_iteration_count=4,  
406         ),  
407     ),  
408     init_state=ArticulationCfg.InitialStateCfg(  
409         pos=(0.0, 0.0, 0.75),  
410         rot=(0.7071, 0, 0, 0.7071),  
411         joint_pos={  
412             ".*_hip_pitch_joint": -0.10,  
413             ".*_knee_joint": 0.30,  
414             ".*_ankle_pitch_joint": -0.20,  
415         },  
416         joint_vel={".*": 0.0},  
417     ),  
418     soft_joint_pos_limit_factor=0.9,  
419     actuators={  
420         "legs": DCMotorCfg(  
421             joint_names_expr=[  
422                 ".*_hip_yaw_joint",  
423                 ".*_hip_roll_joint",  
424                 ".*_hip_pitch_joint",  
425                 ".*_knee_joint",  
426             ],  
427             effort_limit={  
428                 ".*_hip_yaw_joint": 88.0,  
429                 ".*_hip_roll_joint": 88.0,  
430                 ".*_hip_pitch_joint": 88.0,  
431                 ".*_knee_joint": 139.0,  
432             },  
433             velocity_limit={  
434                 ".*_hip_yaw_joint": 32.0,  
435                 ".*_hip_roll_joint": 32.0,  
436                 ".*_hip_pitch_joint": 32.0,  
437                 ".*_knee_joint": 20.0,  
438             },  
439             stiffness={  
440                 ".*_hip_yaw_joint": 100.0,  
441                 ".*_hip_roll_joint": 100.0,  
442                 ".*_hip_pitch_joint": 100.0,  
443                 ".*_knee_joint": 200.0,  
444             },  
445             damping={  
446                 ".*_hip_yaw_joint": 2.5,  
447                 ".*_hip_roll_joint": 2.5,  
448                 ".*_hip_pitch_joint": 2.5,  
449                 ".*_knee_joint": 5.0,  
450             },  
451             armatures={  
452                 ".*_hip_*": 0.03,
```

Retargeting g1

- Then, we compared the AMASS/G1 target positions with the actual G1 link positions.
- Purpose: obtaining the coordinate and scale mismatch.

G1 leg limits

- Ensuring the retargeting doesn't surpass legal G1 leg joint ranges.
- Prints limits for hip pitch/roll/yaw, knee, and ankle pitch/roll

Inverse Kinematics (1 Frame)

- Initially, we solve Inverse Kinematics for one frame only.
- We currently optimize the 12 leg joints of the G1.
- Techniques used are:
 - Fixed root
 - Damped least-squares IK
 - Joint limit clamping

```
Initial error norm: 0.03523858264088631
Initial left foot: [-0.11853916 -0.01996473 -0.0062656 ]
Target left:      [-0.14253801  0.00042049 -0.01309067]
Initial right foot: [ 0.11853986 -0.01996011 -0.00626558]
Target right:      [ 0.10790695 -0.01113793 -0.00984114]

Iter 000 | error norm = 0.035239
Iter 005 | error norm = 0.009739
Iter 010 | error norm = 0.009684
Iter 015 | error norm = 0.009654
Iter 020 | error norm = 0.009679
Iter 025 | error norm = 0.009651
Iter 030 | error norm = 0.009628
Iter 035 | error norm = 0.009657

Final error norm: 0.009586717933416367
Final left foot: [-0.14267416 -0.00452457 -0.00605257]
Target left:      [-0.14253801  0.00042049 -0.01309067]
Final right foot: [ 0.10798129 -0.01357743 -0.00638551]
Target right:      [ 0.10790695 -0.01113793 -0.00984114]

Optimized leg joint angles:
left_hip_pitch_joint      -0.02083
left_hip_roll_joint       +0.01630
left_hip_yaw_joint        -0.09464
left_knee_joint           +0.06109
left_ankle_pitch_joint    -0.03204
left_ankle_roll_joint     +0.00038
right_hip_pitch_joint     -0.00815
right_hip_roll_joint      +0.01928
right_hip_yaw_joint       +0.00767
right_knee_joint          +0.06109
right_ankle_pitch_joint   -0.03009
right_ankle_roll_joint    -0.00005

Saved g1_ik_one_frame_result.npz
```

Inverse Kinematics (100 Frames)

- For each frame:
 - We update from the previous frame (initial pose)
 - We solve the Leg Inverse Kinematics
 - Save the Q trajectory

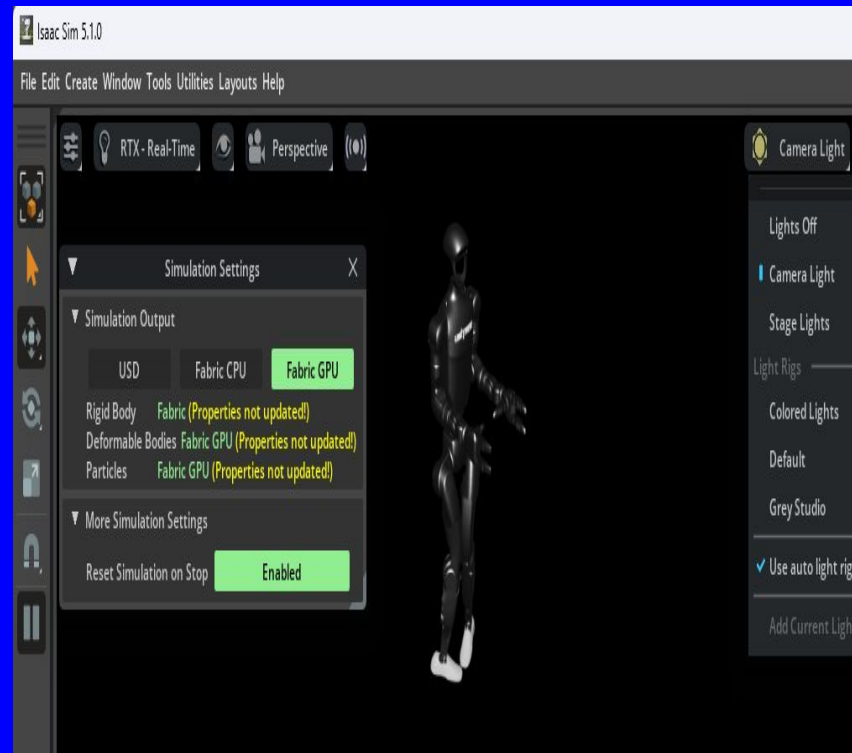
```
Running IK for 100 frames...
```

```
Frame 000 | error norm = 0.009679  
Frame 010 | error norm = 0.002680  
Frame 020 | error norm = 0.000214  
Frame 030 | error norm = 0.000555  
Frame 040 | error norm = 0.000000  
Frame 050 | error norm = 0.000000  
Frame 060 | error norm = 0.000000  
Frame 070 | error norm = 0.000000  
Frame 080 | error norm = 0.000000  
Frame 090 | error norm = 0.000000
```

```
Saved: C:\Users\Mario\Documents\Homework\Internship Stuff\YiSunInternship\amass_g1\g1_leg_ik_clip.npz  
q trajectory shape: (100, 43)  
mean error: 0.0007895079750365852  
max error: 0.009806823916733265  
min error: 1.085500329622846e-07
```

First Stage Inverse Kinematics Controller Results

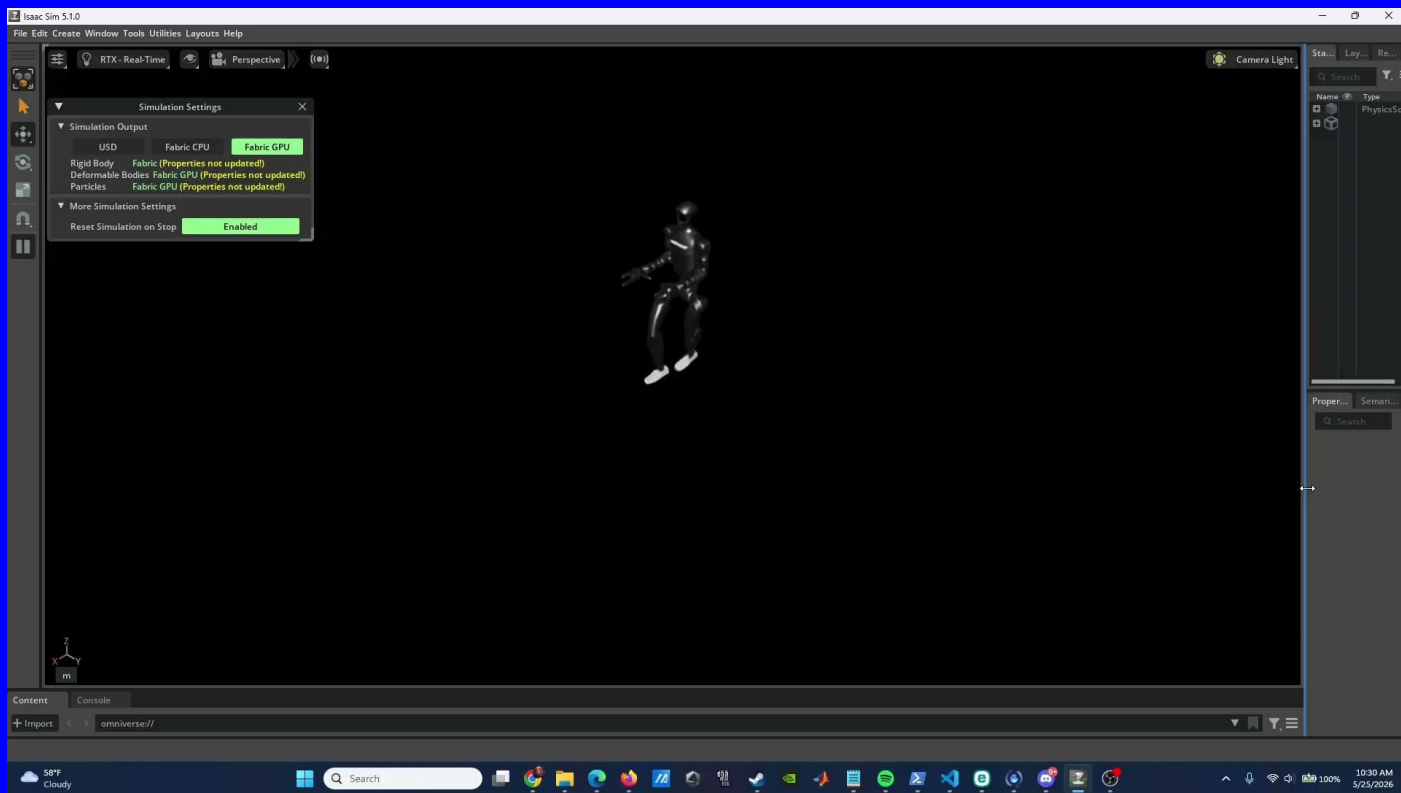
- We play the G1 leg trajectory in Isaac Sim through Isaac Lab
 - Done by applying the $q[t]$ onto the G1 joints
- Visual Results: Robot's Leg + Feet Position slightly move inwards through the 100 frame clip.
- Conclusion: Pipeline of AMASS + SMPL+H $\rightarrow$ G1 Retargeting $\rightarrow$ Inverse Kinematics is proven to work.
- Next Step: In depth Inverse Kinematics



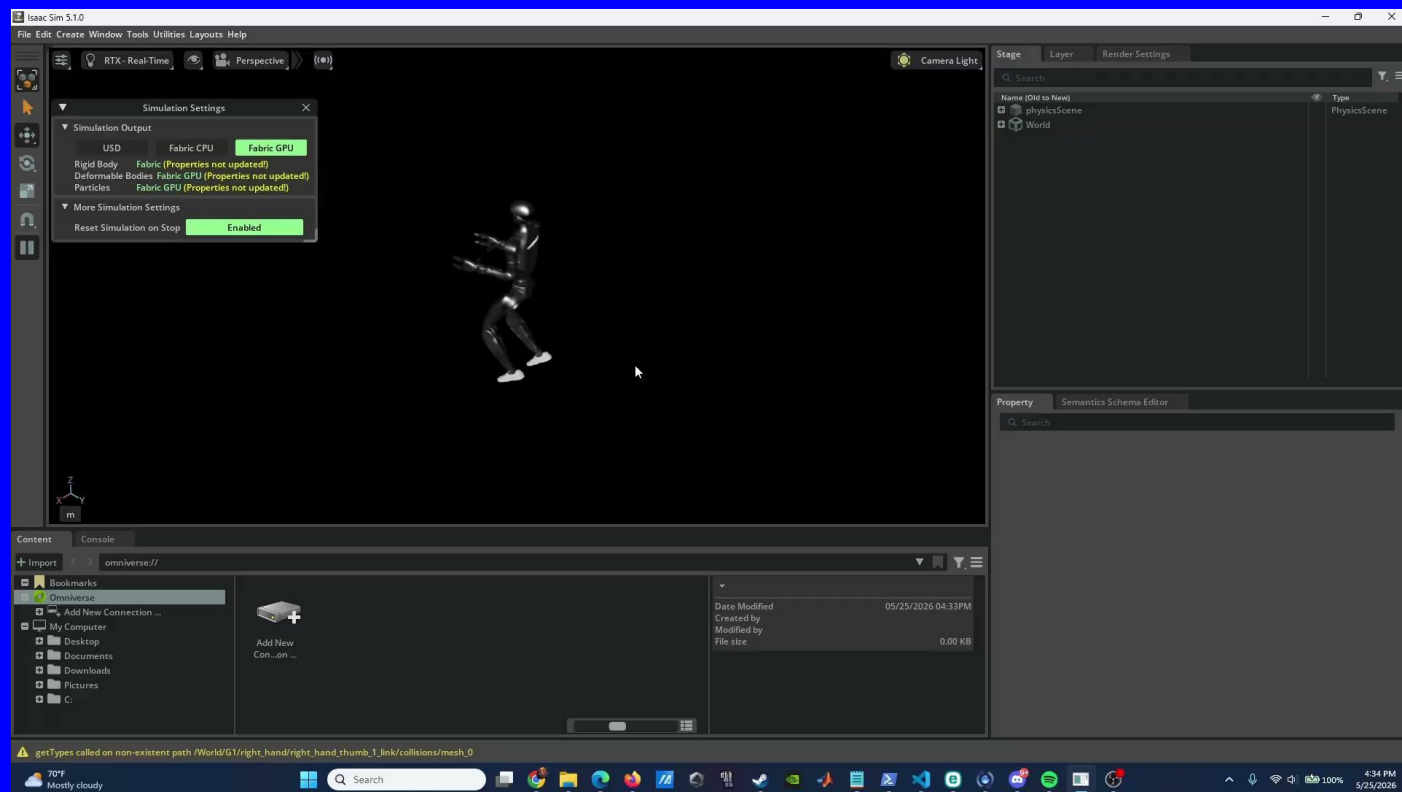
Next Steps

- Now that we have achieved verification of leg movement use Inverse Kinematics, we can implement expand on the joints that are being used.
- We now go from controlling only feet to the entire lower body (pelvis).

More Inverse Kinematics Implementation



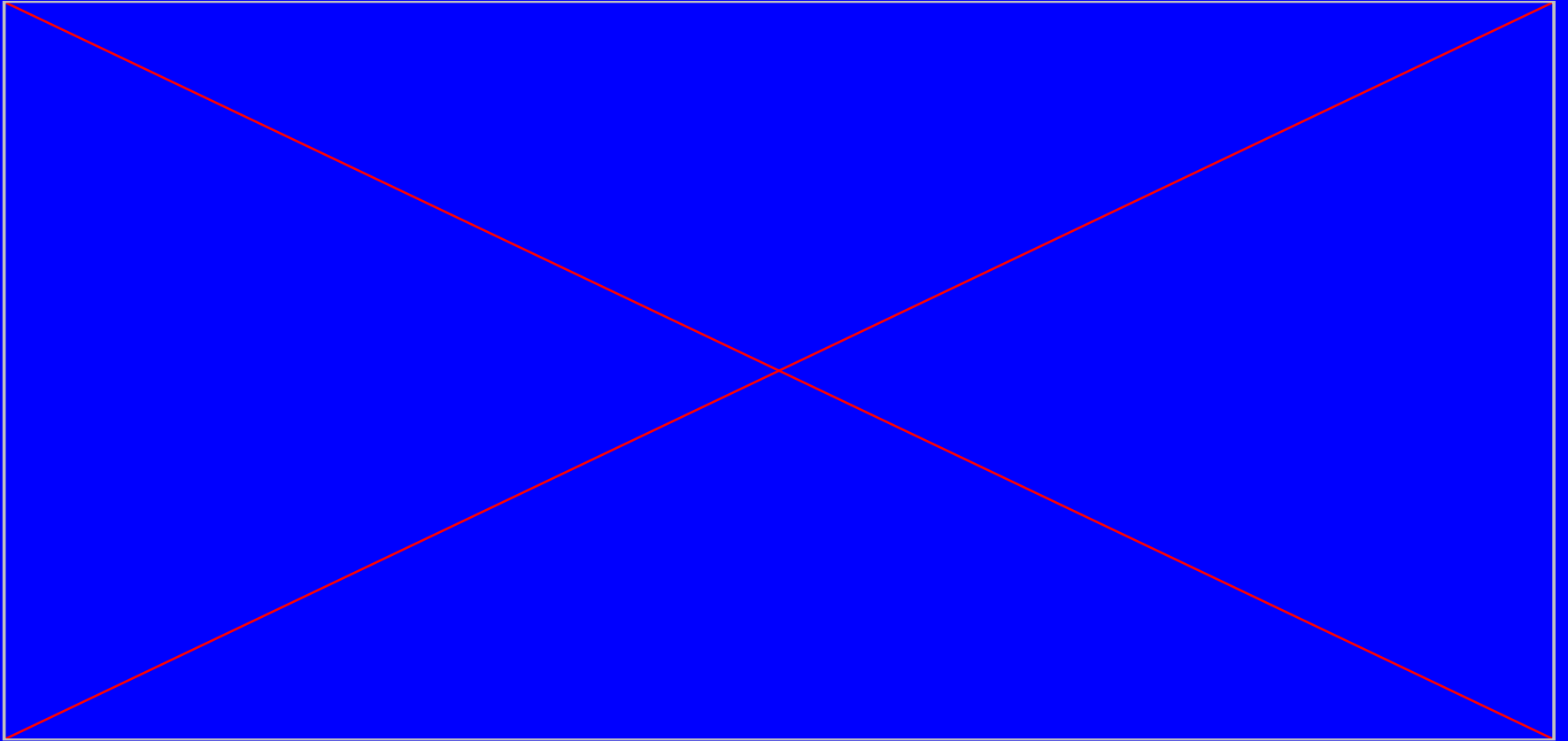
More Inverse Kinematics Implementation (After Tuning)



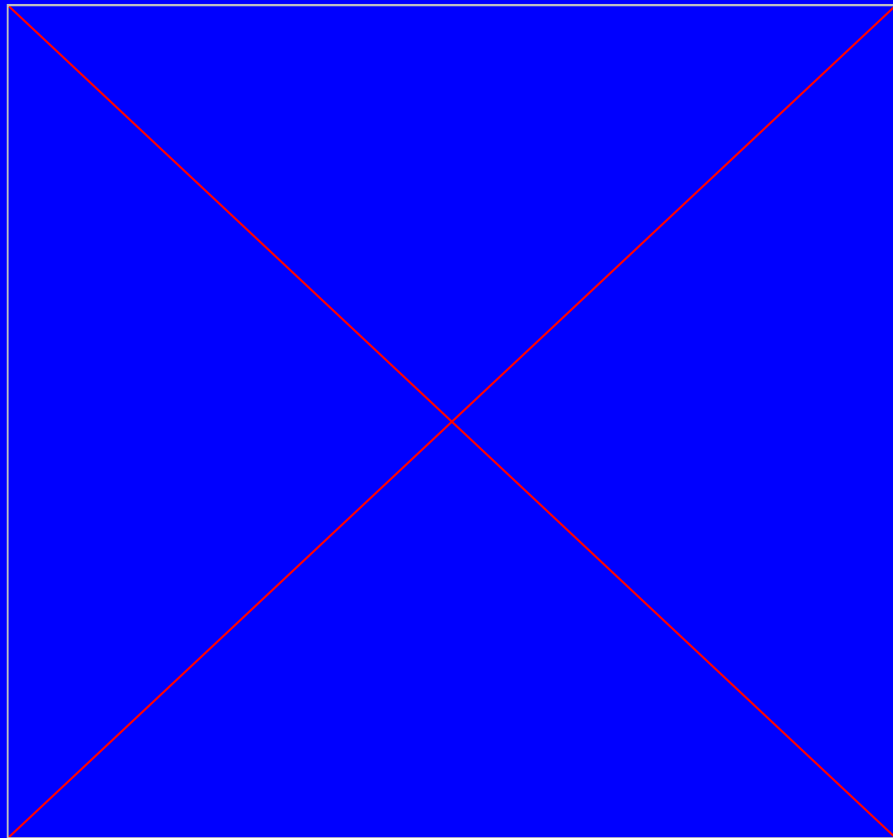
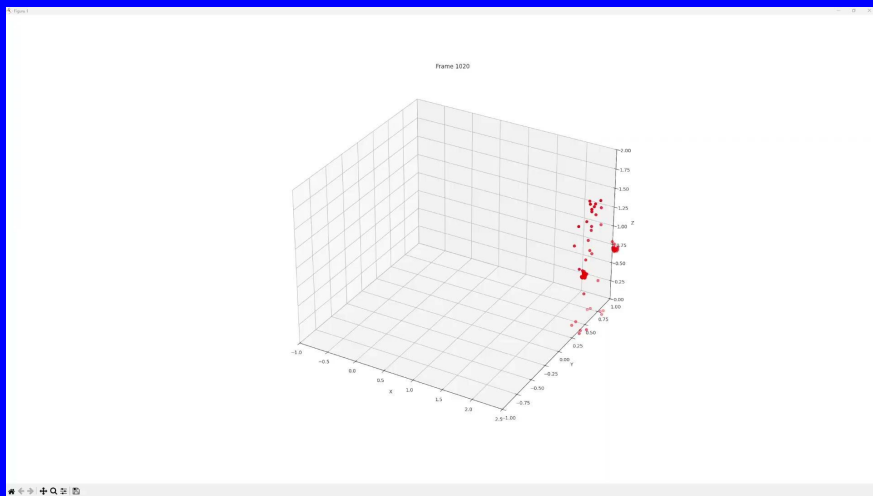
Upper Body Implementation

Next step is implementing Inverse Kinematics Control on the Unitree G1's Arm joints.

More Inverse Kinematics Implementation



Compare *AMASS* dataset with G1



In depth explanation of Inverse Kinematics Retargeting Code

Goal: Convert AMASS human motion dataset into Unitree G1 target positions.

`prepare_g1_targets.py`

- Currently extracts joint data (pelvis/base, left/right foot, left/right hand, head) and then
 1. Recenters motion around the initial pelvis
 2. Scales human motion to G1 size
 3. Converts AMASS coordinates to Isaac Coordinates
 4. Raises Pelvis to G1 standing height

In depth explanation of Inverse Kinematics Retargeting Code

Retarget_g1_ik_clip_root.py

- Solves inverse kinematics for the G1 to track the AMASS-derived motion.
 - Optimizes root position, root yaw, leg joints, waist joints, arm joints. → Matches the G1's pelvis position, left/right foot position, left/right hand position, torso/head proxy.
 - Root position: Comes from the AMASS pelvis target → root yaw is computed from pelvis movement direction.
 - Smoothing: Trajectories are computed before IK (fixes jittery turning)
 - IK Error Function: Computes the error between current G1 link positions and target positions.
 - Total error includes all joint error (pelvis, foot, hand, torso/head, pose regularization)
 - *Pose regularization: encourages robot to have a natural pose.

In depth explanation of Inverse Kinematics Retargeting Code (continued)

Retarget_g1_ik_clip_root.py (continued)

- Jacobian: We use finite differences to build a jacobian (computes the change in task error / change in joint angle)
- Damped Least-Squares IK:
 - Computes a joint update using the function:
 - Prevents instability (reduces exploding joint updates)
- Clamping: each update is clamped to G1's limits to prevent illegal joint positions, knee angles, etc.

$$\Delta = -(J^T J + \lambda I)^{-1} J^T \times error$$

play_g1_ik_clip.py

- Code to load the saved IK trajectory and visualize it on Isaac Sim.

Next Steps

Tuning/Testing

- More tuning of the Inverse Kinematics Control to ensure AMASS dataset is properly retargeted onto the G1.
- Attempt to use this code with other AMASS datasets.

Imitation Learning:

- Using this retargeted Unitree G1 dataset, we can now implement this with Imitation Learning.

References

<https://www.youtube.com/watch?v=g9dBdkwtURE>

<https://amass.is.tue.mpg.de/index.html>

<https://github.com/ZhengyiLuo/SMPLSim>

<https://mano.is.tue.mpg.de/index.html>